

SESSION 09

Sets in Python

What is a Set in Python?

A set is a collection of values where every value is unique. If you try to add a duplicate, it simply gets ignored. Sets are perfect when you care about what is present, but not about order or how many times it appears.

```
# List can have duplicates
emails_list = ["a@gmail.com", "b@gmail.com", "a@gmail.com"]
print(f"List with duplicates: {emails_list}")
# Output: ['a@gmail.com', 'b@gmail.com', 'a@gmail.com']

# Set automatically removes duplicates
emails_set = {"a@gmail.com", "b@gmail.com", "a@gmail.com"}
print(f"Set after removing duplicates: {emails_set}")
# Output: {'a@gmail.com', 'b@gmail.com'} (order may vary)
```

This example clearly demonstrates how sets maintain only unique elements, making them ideal for tasks like filtering out redundant data.

Key Properties of Sets

Unordered

There is no concept of a first or last element; the order of elements is not guaranteed and can change.

Unique Values Only

Sets inherently store only distinct elements, automatically discarding any duplicates added.

Mutable

You can add or remove items from a set after it has been created.

No Indexing

Elements cannot be accessed by their position (index) because sets are unordered.

Immutable Elements

The elements within a set must be immutable (e.g., strings, numbers, tuples). Lists or other sets cannot be elements of a set.

Sets are highly efficient for operations like checking if a value exists, removing duplicates from a collection, or performing mathematical set operations such as unions and intersections. They are a fundamental data structure for tasks where membership testing and uniqueness are paramount.

Creating a Set

Sets in Python can be initialised in two primary ways: using curly braces `{}` for non-empty sets, or by explicitly calling the `set()` constructor, especially for creating empty sets or converting other iterables.

```
# Set with string values
fruits = {"apple", "mango", "banana"}
print(f"Fruits set: {fruits}") # Output: {'banana', 'apple', 'mango'} (order may vary)

# Set with numbers
nums = {1, 2, 3, 4, 5}
print(f"Numbers set: {nums}") # Output: {1, 2, 3, 4, 5}

# Set with mixed data types
mixed = {"Rahul", 21, True, 9.5}
print(f"Mixed set: {mixed}") # Output: {True, 9.5, 'Rahul', 21} (order may vary)

# Creating a set from a list, automatically removing duplicates
my_list = [1, 2, 3, 2, 1, 4, 5, 4]
unique = set(my_list)
print(f"Unique elements from list: {unique}") # Output: {1, 2, 3, 4, 5}
```

These examples showcase the flexibility of creating sets from various data types and iterables, highlighting their automatic duplicate removal feature.

Empty Set – The Gotcha

One crucial distinction for beginners is how to correctly create an empty set, as curly braces alone default to an empty dictionary.

```
# This is NOT a set — it's an empty dictionary
empty_braces = {}
print(f"Type of {}: {type(empty_braces)}") # Output: <class 'dict'>

# This IS an empty set
empty_set_func = set()
print(f"Type of set(): {type(empty_set_func)}") # Output: <class 'set'>
```

⊗ **Watch Out:** To create an empty set, you must use `set()`. Writing `{}` gives you an empty dictionary, not a set. This is a very common beginner mistake that can lead to unexpected behaviour in your code.

Why Sets Don't Allow Duplicates

The fundamental characteristic of a set in Python is that it only stores unique values. When you attempt to add an element that already exists, Python simply ignores the duplicate without raising an error or warning you. This makes sets incredibly efficient for ensuring data uniqueness.

```
nums = {10, 20, 30, 20, 10, 40}
print(nums)
# Output: {10, 20, 30, 40}    # Duplicates removed automatically
```

As observed, even though `10` and `20` were listed multiple times during initialisation, the resulting set `nums` contains each number only once.

```
# A practical use — removing duplicates from a list
emails = ["a@gmail.com", "b@gmail.com", "a@gmail.com", "c@gmail.com"]
unique_emails = list(set(emails))
print(unique_emails)
# Output: ['a@gmail.com', 'b@gmail.com', 'c@gmail.com']
# (Note: The order may vary since sets are inherently unordered)
```

This demonstrates a common use case: quickly filtering out duplicate entries from any iterable, like a list of email addresses. Converting to a set and then back to a list is a concise way to achieve this.

```
# Counting unique words in a sentence
sentence = "python is great and python is easy"
unique_words = set(sentence.split())
print(len(unique_words))    # Output: 5
print(unique_words)         # Output: {'python', 'is', 'great', 'and', 'easy'}
```

Here, sets effectively count distinct words in a sentence, showcasing their utility in text processing and data analysis tasks where uniqueness is key.

 **Quick Tip:** Whenever you need to remove duplicates from any collection, converting it to a set is often the fastest and most Pythonic one-line solution.

Accessing Elements (Membership Check)

Since sets are unordered, direct access to elements using an index is not possible. Instead, you can efficiently verify if a value exists within a set using the `in` and `not in` operators.

```
fruits = {"apple", "mango", "banana"}

# This will raise a TypeError: 'set' object is not subscriptable
# print(fruits[0])

print("apple" in fruits)    # Output: True
print("grapes" in fruits)   # Output: False
print("kiwi" not in fruits) # Output: True
```

This method is particularly powerful for scenarios requiring quick verification of an item's presence.

```
# Practical use — checking if a user has access
allowed_users = {"rahul", "priya", "karan"}

user = input("Enter username: ")
if user in allowed_users:
    print("Access granted.")
else:
    print("Access denied.")
```

 **Note:** Membership checks (`in`) are exceptionally fast with sets, significantly outperforming similar operations on lists. This efficiency is a primary reason sets are preferred for lookups in real-world applications.

Adding and Removing Elements

Sets are mutable, meaning you can add new elements or remove existing ones after creation. Python provides several methods for these operations.

Adding Elements

Use `add()` for single elements, and `update()` for multiple elements from an iterable. Duplicates are silently ignored.

```
pythonfruits = {"apple", "mango"}

# add() — adds a single element
fruits.add("banana")
print(fruits)  # {'apple', 'mango', 'banana'}

# update() — adds multiple elements (from any iterable)
fruits.update(["grapes", "kiwi"])
print(fruits)  # {'apple', 'mango', 'banana', 'grapes', 'kiwi'}

# Adding a duplicate is silently ignored
fruits.add("apple")
print(fruits)  # no change
```

Removing Elements

Python offers methods like `remove()`, `discard()`, `pop()`, and `clear()` to manage set elements.

```
pythonfruits = {"apple", "mango", "banana", "kiwi"}

# remove() — removes the value, raises KeyError if not found
fruits.remove("mango")
print(fruits)  # {'apple', 'banana', 'kiwi'}

# fruits.remove("grapes")  # KeyError: 'grapes'

# discard() — removes if present, no error if not found (safer)
fruits.discard("banana")  # works
fruits.discard("grapes")  # silently does nothing

# pop() — removes and returns a random element
removed = fruits.pop()
print(removed)  # any element

# clear() — removes everything
fruits.clear()
print(fruits)  # set()
```

 **Quick Tip:** Use `discard()` when you are not sure whether the element exists. Use `remove()` only when you are sure — or when you want an error to catch a bug.

Set Operations: Union and Intersection

Sets are powerful tools for performing mathematical set operations. The two most common operations are **union** and **intersection**, which allow you to combine or find common elements between sets.

Union (`|` or `.union()`)

The union of two sets includes all unique elements present in either set, effectively combining them without duplicates.

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

# Using the | operator
print(a | b)      # {1, 2, 3, 4, 5, 6}

# Using the union() method
print(a.union(b)) # {1, 2, 3, 4, 5, 6}
```

Intersection (`&` or `.intersection()`)

The intersection of two sets consists of only the elements that are common to both sets.

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

# Using the & operator
print(a & b)      # {3, 4}

# Using the intersection() method
print(a.intersection(b)) # {3, 4}
```

Real-world Application: Student Records

Consider two sets representing students enrolled in different subjects. Set operations can easily identify students taking both or any of the subjects.

```
math_students = {"Rahul", "Priya", "Karan", "Sana"}
science_students = {"Priya", "Karan", "Aman"}

print("Students in BOTH Math and Science:")
print(math_students & science_students)
# Output: {'Priya', 'Karan'}

print("\nStudents in EITHER Math OR Science:")
print(math_students | science_students)
# Output: {'Rahul', 'Priya', 'Karan', 'Sana', 'Aman'}
```

Note: Both the operator (`|`, `&`) and the method (`.union()`, `.intersection()`) achieve the same result. Choose the one that enhances readability for your specific use case. The methods can also take multiple iterables as arguments.

Looping Through a Set

Although sets are unordered collections, you can easily iterate over their elements using a `for` loop. This allows you to process each unique item within the set.

```
fruits = {"apple", "mango", "banana"}

# Simple for loop
for fruit in fruits:
    print(fruit)

# Output (order may vary — sets are unordered):
# mango
# apple
# banana
```

While less common, you can also use `enumerate`, though it doesn't offer much practical benefit for sets given their unordered nature.

```
# Looping with enumerate (rarely needed for sets, but works)
for i, fruit in enumerate(fruits, start=1):
    print(f"{i}. {fruit}")
```

Practical Application: Unique Words

A powerful use case is extracting and processing unique items, like words from a sentence, ensuring each word is handled only once.

```
# Practical use — printing only unique words from a sentence
sentence = "python is fun and python is powerful"
words = set(sentence.split())

for word in words:
    print(word)

# Output (order may vary):
# is
# powerful
# fun
# python
# and
```

 **Watch Out:** The order in which elements are printed may be different each time you run the program. Sets do not maintain any inherent order; therefore, you should never rely on it for sequential processing.

Built-in Functions on Sets

Python provides several useful built-in functions to easily perform common operations on sets, such as determining size, finding extremes, and calculating sums.

```
nums = {40, 10, 30, 20, 50}

print(len(nums)) # Output: 5 (number of elements)
print(min(nums)) # Output: 10 (smallest element)
print(max(nums)) # Output: 50 (largest element)
print(sum(nums)) # Output: 150 (sum of all elements)
```

The `sorted()` function returns a **new list** containing all elements from the set in sorted order. It does not modify the original set.

```
nums = {40, 10, 30, 20, 50}

print(sorted(nums))          # Output: [10, 20, 30, 40, 50] (ascending)
print(sorted(nums, reverse=True)) # Output: [50, 40, 30, 20, 10] (descending)
```

Sets can also be easily converted to and from other iterable types like lists and tuples, facilitating data manipulation.

```
my_list = [1, 2, 3, 2, 1]
my_set = set(my_list)      # Converts list to set: {1, 2, 3}

back_list = list(my_set)   # Converts set back to list: [1, 2, 3]
my_tuple = tuple(my_set)   # Converts set to tuple: (1, 2, 3)
```

This allows for powerful data processing workflows, such as extracting unique, sorted values from a list:

```
marks = [85, 90, 85, 78, 92, 90, 78]
unique_sorted = sorted(set(marks)) # First unique, then sort
print(unique_sorted)               # Output: [78, 85, 90, 92]
```

 **Note:** `sorted()` always returns a list. If you need the result to be a set again, the data is already deduplicated, so a list is usually the desired output anyway.

Set vs. List vs. Tuple: Key Differences

Understanding the distinctions between Python's core collection types — lists, tuples, and sets — is crucial for writing efficient and reliable code. Each serves a unique purpose based on its characteristics.

Feature	List	Tuple	Set
Syntax	[1, 2, 3]	(1, 2, 3)	{1, 2, 3}
Ordered	Yes	Yes	No
Mutable	Yes	No	Yes
Allows Duplicates	Yes	Yes	No
Indexing	Yes	Yes	No
Use Case	Ordered, changing data	Fixed, unchanging data	Unique values, fast lookup

When to Use Each

When to use a List:

- You need ordered data where insertion order matters.
- Duplicates are allowed and necessary.
- Items will frequently change (add, remove, modify).

Example: A shopping cart, a list of student names, transaction history.

When to use a Tuple:

- Data should not change after creation (immutable).
- You want to protect data from accidental modification.
- Used as dictionary keys (since they are hashable).

Example: Geographic coordinates (latitude, longitude), RGB colour values, database records that shouldn't be altered.

When to use a Set:

- You need a collection of only unique values.
- The order of elements does not matter.
- You need very fast membership checks (in keyword).

Example: Removing duplicates from a list, tracking allowed users, storing tags for a post, performing mathematical set operations.

Quick Demo

```
# Quick demo of all three
items_list = ["apple", "mango", "apple"] # keeps order, keeps duplicates
items_tuple = ("apple", "mango", "apple") # keeps order, immutable
items_set = {"apple", "mango", "apple"} # unordered, removes duplicates

print(f'List: {items_list}') # Output: List: ['apple', 'mango', 'apple']
print(f'Tuple: {items_tuple}') # Output: Tuple: ('apple', 'mango', 'apple')
print(f'Set: {items_set}') # Output: Set: {'apple', 'mango'} (order may vary)
```